Recursividade

Estrutura de Dados

Universidade Federal de Juiz de Fora
Departamento de Ciência da Computação

Conteúdo

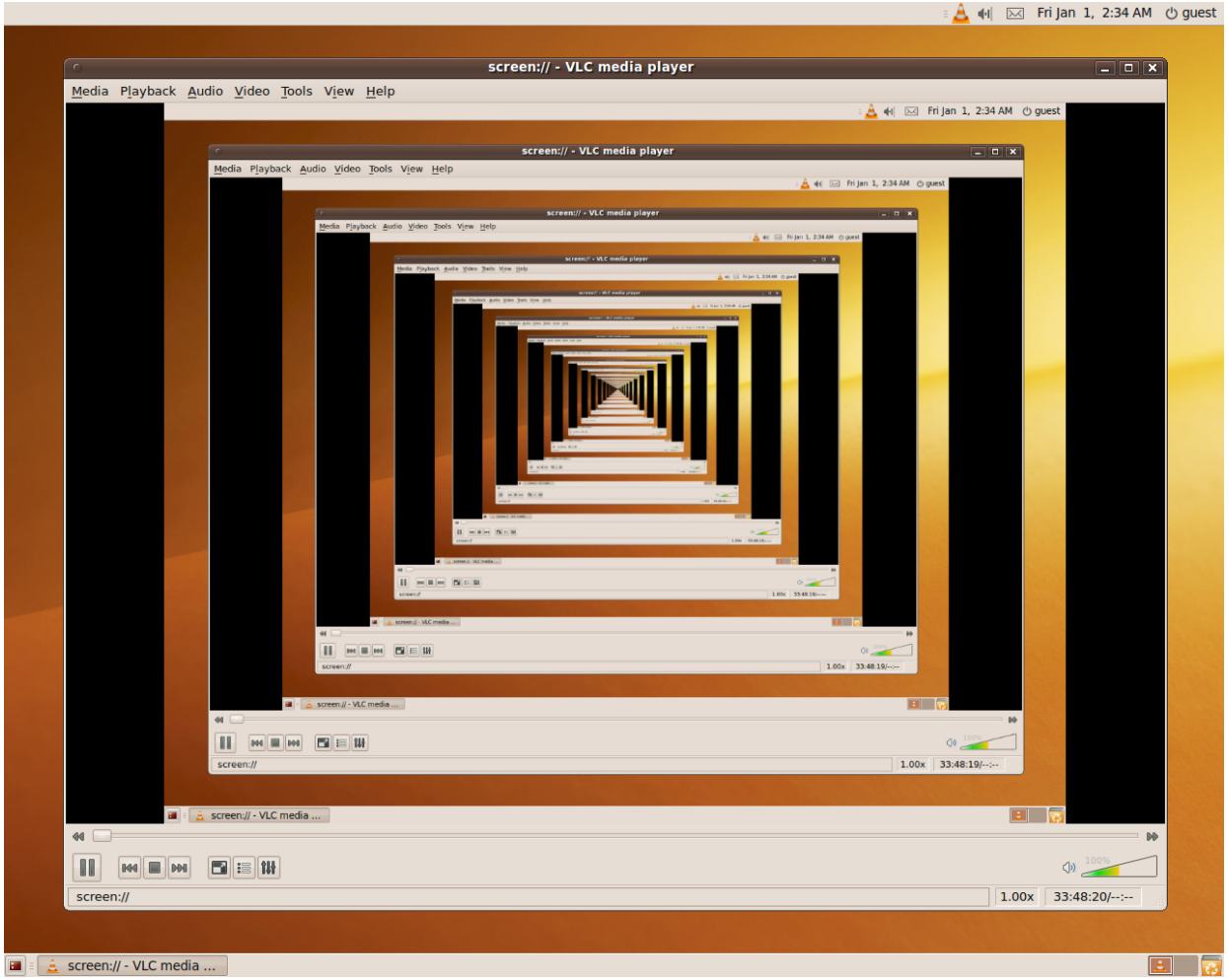
- Recursividade
- Definição
- Algoritmos recursivos
- Exemplos

Recursividade

- Recursão é uma abordagem de solução de problemas que pode ser utilizada para gerar soluções simples a certos tipos de problemas que seriam difíceis de resolver de outra maneira.
- Em um algoritmo recursivo, o problema original é dividido e um ou mais versões simples de si mesmo.
- P. ex., se a solução do problema original envolve n itens, o pensamento recursivo deve dividi-lo em 2 problemas: um envolvendo $n - 1$ itens e um outro envolvendo um único item.
- O problema com $n - 1$ itens poderia ser dividido novamente em um envolvendo $n - 2$ itens e um outro envolvendo apenas um único item, e assim por diante.
- Se a solução de todos os problemas com um item é trivial, podemos construir a solução do problema original a partir das soluções dos problemas mais simples.

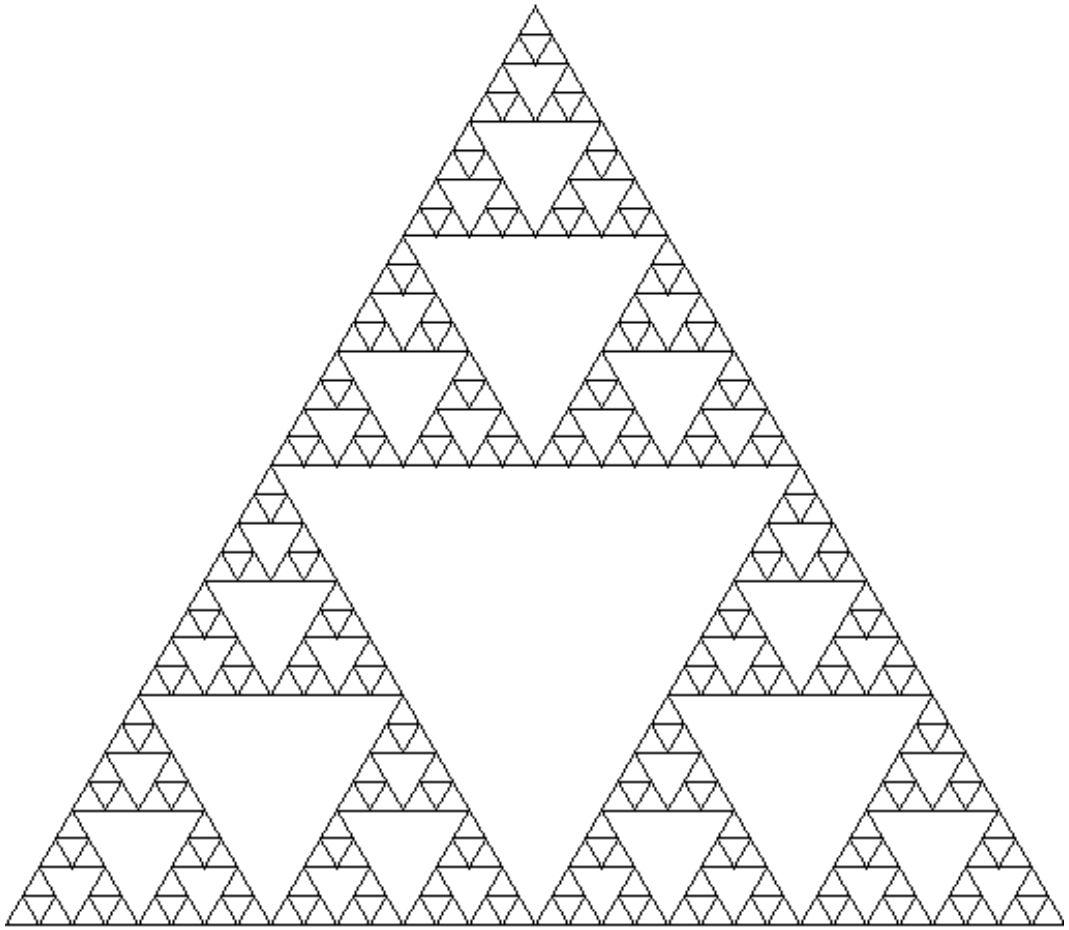
Recursividade

- Em imagens...



Recursividade

- Em imagens...



Recursividade

Algoritmo recursivo genérico

- Considere que n seja um inteiro que representa, de alguma forma, o tamanho de um determinado problema que deseja-se resolver utilizando um algoritmo recursivo.
- Descrição geral de um algoritmo recursivo:

1. Se o problema puder ser resolvido diretamente para o valor atual de n
   - **Resolva-o**
2. Senão
   - Aplique **recursivamente** o algoritmo a um ou mais problemas (iguais ao original), porém envolvendo valores menores de n.
3. **Combine** as soluções dos problemas menores para obter a solução do problema original.

Recursividade

Algoritmo recursivo genérico

- No `se` tem-se um teste para o que é chamado de **caso base**: o valor de n para o problema é suficientemente pequeno e, portanto, o problema pode ser resolvido facilmente.
- No `senão`, tem-se o **passo recursivo**, porque aqui aplica-se o algoritmo recursivamente, isto é, para resolver o mesmo problema, mas para uma instância menor (valor de n menor).
- Sempre que ocorrer uma divisão, revisita-se o caso base para cada novo problema a fim de verificar se esse é um caso base ou um caso recursivo.

Recursividade

- Uma solução recursiva tem as seguintes características:
  - Deve-se conhecer a solução direta para o caso base (para um valor pequeno de n)
  - Um problema de um dado tamanho (digamos, n) pode ser dividido em uma ou mais versão menores do mesmo problema (caso recursivo).
- Será visto, daqui em diante, exemplos de soluções recursivas em C++ para alguns problemas comuns de programação.
  - Fatorial
  - Sequência de Fibonacci
  - Máximo divisor comum
  - etc
- Note que, em implementações recursivas, as funções que resolvem um determinado problema devem chamar a si mesmo.

Recursividade

Fatorial

- **Definição:** o fatorial de um inteiro não-negativo n, representado por $n!$, é o produto de todos os inteiros positivos menores ou iguais a n.
- Adota-se por definição que $0! = 1$.
- Exemplos:
  - $0! = 1$
  - $1! = 1$
  - $2! = 2 \times 1$
  - $3! = 3 \times 2 \times 1$
  - $4! = 4 \times 3 \times 2 \times 1$
  - $5! = 5 \times 4 \times 3 \times 2 \times 1$
- Note que
  - $5! = 5 \times 4!$

Recursividade

Fatorial

- Implementação **iterativa** (não-recursiva)

```
int fatorial(int n)
{
  int fat = 1;
  for(int i=1; i<=n; i++)
    fat = fat * i;
  return fat;
}
```

- Implementação **recurvisa**

```
int fatorial(int n)
{
  if(n==0 || n==1)
    return 1;
  else
    return n * fatorial(n-1);
}
```

Recursividade

Fatorial

fatorial(4) → fatorial(3) → fatorial(2) → fatorial(1)

```
int fatorial(int n)
{
  if(n==0 || n==1)
    return 1;
  else
    return n * fatorial(n-1);
}
```

n=4; 4; 6; 24

```
int fatorial(int n)
{
  if(n==0 || n==1)
    return 1;
  else
    return n * fatorial(n-1);
}
```

n=3; 3; 2

```
int fatorial(int n)
{
  if(n==0 || n==1)
    return 1;
  else
    return n * fatorial(n-1);
}
```

n=2; 2; 1

```
int fatorial(int n)
{
  if(n==0 || n==1)
    return 1;
  else
    return n * fatorial(n-1);
}
```

n=1; 1

Recursividade

Potência

- Desenvolver uma função recursiva para calcular x^n.
- Para simplificar, inicialmente, considere que $n \geq 0$.
- Definição recursiva do problema:

$$x^n = \begin{cases} 1, & \text{se } n = 0 \\ xx^{n-1}, & \text{se } n > 0 \end{cases}$$

Recursividade

Potência

- Implementação iterativa

```
float pot(float x, int n)
{
  float r=1.0;
  for(int i=1; i<=n; i++)
    r = r * x;
  return r;
}
```

- Implementação **recursiva**

```
float pot(float x, int n)
{
  if(n==0)
    return 1.0;
  else
    return x * pot(x,n-1);
}
```

Recursividade

Potência

- Se n pode ser negativo, então

$$x^n = \begin{cases} 1, & \text{se } n = 0 \\ \frac{1}{x^n}, & \text{se } n < 0 \\ xx^{n-1}, & \text{se } n > 0 \end{cases}$$

- E assim

```
float pot(float x, int n)
{
  if(n==0)
    return 1.0;
  else if(n < 0)
    return 1.0/pot(x,-n);
  else
    return x * pot(x,n-1);
}
```

Recursividade

Potência

- Pode-se ainda pensar na seguinte melhoria dessa função:

$$x^n = \begin{cases} 1, & \text{se } n = 0 \\ \frac{1}{x^n}, & \text{se } n < 0 \\ (x^2)^{\lfloor n/2 \rfloor}, & \text{se } n \text{ é par} \\ x(x^2)^{\lfloor n/2 \rfloor}, & \text{se } n \text{ é ímpar} \end{cases}$$

```
float pot(float x, int n)
{
  if(n==0)
    return 1.0;
  else if(n < 0)
    return 1.0/pot(x,-n);
  else if(n%2==0)
    return pot(x*x,n/2);
  else
    return x*pot(x*x,n/2);
}
```

Recursividade

MDC

- Calcular o máximo divisor comum (m.d.c.) entre m e n.
- Definição recursiva:

$$mdc(m,n) = \begin{cases} mdc(n,m), & \text{se } M < n \\ n, & \text{se } n \text{ é divisor de } m \\ mdc(n, m \bmod n), & \text{caso contrário} \end{cases}$$

```
int mdc(int m, int n)
{
  if(m < n)
    return mdc(n, m);
  else if(m%n == 0) // caso base - conhecido
    return n;
  else              // passo recursivo
    return mdc(n, m%n);
}
```

Recursividade

Fibonacci

- A sequência de Fibonacci é uma sequência de números inteiros, que começa com os números 0 e 1, na qual cada termo subsequente corresponde à soma dos dois anteriores.

$$0, 1, \mathbf{1}, \mathbf{2}, \mathbf{3}, \mathbf{5}, \mathbf{8}, \mathbf{13}, \mathbf{21}, \mathbf{34}, \mathbf{55}, \mathbf{89}, \mathbf{144}, \mathbf{233}, \mathbf{377}, \mathbf{610}, \mathbf{987}, \ldots$$

- Seja $F_1 = 1$ o primeiro número. Se F_n é o n-ésimo número da sequência de Fibonacci, então, este pode ser definido como:

$$F_n = F_{n-1} + F_{n-2}$$

Recursividade

Fibonacci

- Implementação **recursiva**

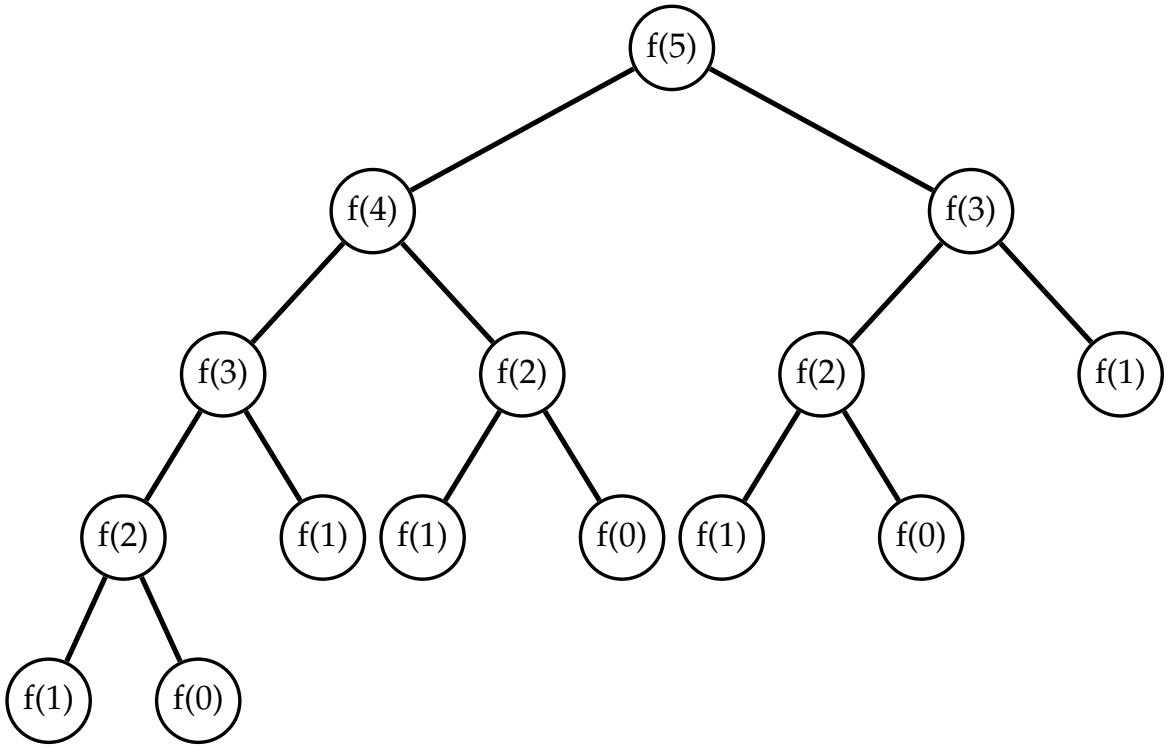
```
int fib(int n)
{
  if(n == 0 || n == 1)
    return 1;
  else
    return fib(n-1) + fib(n-2);
}
```

- Note que no `else` dessa implementação recursiva, a função `fib` é chamada duas vezes.

Recursividade

Fibonacci

- Essa solução é muito ineficiente.
- Exemplo para $n = 5$ (foi usado f(n) no lugar de fib(n)):



- O problema com essa solução é que a função é chamada várias vezes para o mesmo valor (parâmetro). Ex: $F(2)$ é calculado 3 vezes.
- Para valores n maiores, a situação piora.

Recursividade

Fibonacci

- Uma outra implementação **recursiva**:

```
int fib(int atual, int anterior, int n)
{
  if(n == 1)
    return atual;
  else
    return fib(atual + anterior, atual, n-1);
}
```

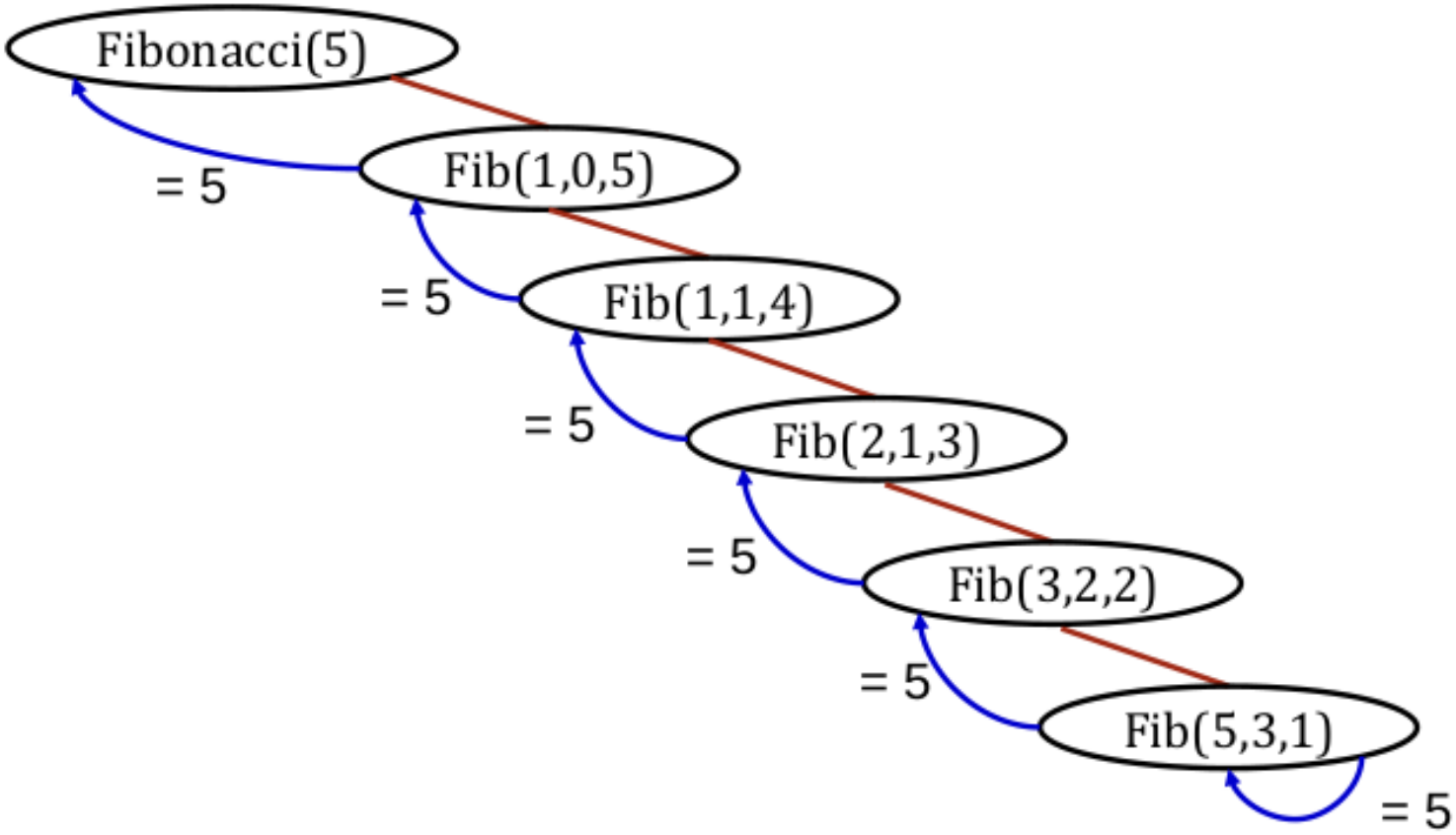
- Para começar a execução deve-se usar a seguinte função

```
int fibonacci(int n)
{
  return fib(1, 0, n);
}
```

Recursividade

Fibonacci

- Essa versão é mais eficiente. Veja o exemplo para $n = 5$.



Recursividade

Intervalo

- Desenvolver uma função recursiva para imprimir todos os números inteiros no intervalo fechado de a até b.

```
void seq(int a, int b)
{
  if(a <= b)
  {
    cout << a << " ";
    seq(a+1, b);
  }
}
```

- Exemplo de como a função deve ser chamada

```
seq(0, 10)
```

- Saída

```
0 1 2 3 4 5 6 7 8 9 10
```

Recursividade

Intervalo

- Qual o resultado de inverter as seguintes instruções?

```
void seq(int a, int b)
{
  if(a <= b)
  {
    seq(a+1, b);
    cout << a << " ";
  }
}
```

Recursividade

Número primo

- Um número primo só é divisível por 1 e por ele mesmo.

```
bool ehPrimo(int p)
{
  return auxPrimo(p, 2);
}

bool auxPrimo(int p, int i)
{
  if (i==p)
    return true;

  if (p%i == 0)
    return false;

  return auxPrimo(p, i+1);
}
```

Recursividade

Maior

- Desenvolver uma função recursiva para calcular e retornar o maior valor de um vetor v com n números reais.
- Caso base: vetor com apenas 1 elemento, que é o maior.
- Passo recursivo: o maior elemento do vetor é ou o último elemento ou o maior elemento dentre os $n-1$ primeiros elementos do vetor.
- Isto é:

$$\max(v, n) = \begin{cases} v[0], & \text{se } n = 1 \\ v[n-1], & \text{se } v[n-1] > \max(v, n-1) \\ \max(v, n-1), & \text{caso contrário} \end{cases}$$

Recursividade

Maior

- Implementação **recursiva**

```
float max(float* vet, int n)
{
  if (n == 1)                  // caso base
    return vet[0];

  float x = max(vet, n-1); // passo recursivo

  if(x > vet[n-1])
    return x;
  else
    return vet[n-1] ;
}
```

Recursividade

Maior

- Outra implementação
- Função empacotadora

```
float max(float* vet, int n)
{
  return maxR(vet, 0, n);
}
```

- Função recursiva

```
float maxR(float* vet, int i, int n)
{
  if(i == n-1)
    return vet[i];

  float x = maxR(vet, i+1, n);

  return (x > vet[i]) ? x : vet[i];
}
```

Recursividade

Pares

- Desenvolver uma função recursiva para calcular e retornar a quantidade de valores pares de um vetor `v` com n números inteiros.

```
int pares(int* vet, int n)
{
  if(n == 1) // caso base
    if (vet[0] % 2 == 0)
      return 1;

  int x = pares(vet, n-1) ; // passo recursivo

  if (vet[n-1] % 2 == 0)
    return x+1;
  else
    return x;
}
```

Exercícios

1. Desenvolver uma função recursiva que recebe um número inteiro n e retorna o valor do somatório:
   $n + (n - 1) + (n - 2) + \ldots + 2 + 1$.
2. Desenvolver uma função recursiva que recebe um vetor de reais e seu tamanho n, calcular e retornar o menor valor do vetor.
3. Desenvolver uma função recursiva que recebe um vetor de reais e seu tamanho n, calcular e retornar a soma de todos os seus valores.
4. Desenvolver uma função recursiva para calcular e retornar a quantidade de valores pares de um vetor `v` com n números inteiros.
5. Desenvolver uma função recursiva para calcular e retornar uma string de caracteres contendo '0' e '1' correspondente a versão binária de um número inteiro positivo dado.